

Projeto Final: Sistemas Operacionais

**Camila Fernanda e S. Lima, Daniel M. Ribeiro, Henrique R. Ribeiro,
João Vitor M. Gomes, Lael K. Hayashi, Lucas A. Gazale**

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
CEP 12.247-014 – São José dos Campos – SP – Brasil

camila.lima@unifesp.br, dmribeiro@unifesp.br, henrique.rodrigues06@unifesp.br
jvmgomes@unifesp.br, lael.hayashi@unifesp.br, lucas.gazale@unifesp.br

Abstract. *This report presents the implementation and evaluation of process scheduling algorithms in the MINIX operating system, developed as part of the Operating Systems course at the Federal University of São Paulo. The objective was to analyze the behavior and performance of different scheduling strategies through modifications to the system's scheduling mechanisms. The algorithms were implemented and subjected to experimental testing, allowing the observation of their characteristics, advantages, and limitations. The results were analyzed using graphical representations, enabling a comparison of the studied approaches and contributing to a better understanding of the challenges involved in process management within operating systems.*

Resumo. *Este relatório apresenta a implementação e avaliação de algoritmos de escalonamento de processos no sistema operacional MINIX, desenvolvida na disciplina de Sistemas Operacionais da Universidade Federal de São Paulo. O objetivo foi analisar o comportamento e o desempenho de diferentes estratégias de escalonamento por meio da modificação dos mecanismos do sistema. Os algoritmos foram implementados e submetidos a testes experimentais, permitindo a observação de suas características, vantagens e limitações. Os resultados foram analisados por meio de representações gráficas, possibilitando uma comparação entre as abordagens estudadas e contribuindo para a compreensão dos desafios envolvidos no gerenciamento de processos.*

1. Introdução

O desenvolvimento dos Sistemas Operacionais esteve historicamente vinculado ao debate entre duas arquiteturas predominantes: monolítica e baseada em microkernels [Tanenbaum and Bos 2016]. Sistemas monolíticos executam todos os serviços sob o mesmo espaço de endereçamento com privilégios de supervisor, o que, apesar de sua ampla adoção, introduz vulnerabilidades estruturais onde a falha de qualquer rotina pode causar colapso total do sistema [Tanenbaum and Woodhull 2006]. Em contrapartida, o MINIX 3 fundamenta-se na filosofia de microkernel, mantendo um núcleo minimalista com apenas os mecanismos indispensáveis, enquanto drivers e servidores operam como componentes isolados em modo usuário — de forma que falhas sejam contidas localmente, sem comprometer o sistema inteiro [Tanenbaum and Bos 2016].

Diante dessa arquitetura, este projeto explora a flexibilidade do MINIX 3 em ambiente virtualizado Oracle VM VirtualBox, modificando seus subsistemas internos para implementar e avaliar três políticas clássicas de escalonamento de CPU em espaço de usuário: Round-Robin, Escalonamento Garantido e Escalonamento por Loteria — cada uma desenvolvida em branch separada do clone do repositório do MINIX 3 (<https://github.com/dmribeiro0/minix>).

A demonstração prática das modificações realizadas no kernel do MINIX 3 encontra-se disponível em um vídeo no YouTube (não listado), acessível pelo link: https://youtu.be/z_6ET-zEd2Y.

2. Etapa 1

2.1. Instalação e Configuração do Minix

A instalação do sistema operacional Minix 3.0 foi realizada em uma máquina virtual utilizando o VirtualBox. Durante o processo, foram encontradas algumas dificuldades, que exigiram ajustes e, em um caso, a reinstalação completa do sistema. Inicialmente, durante a etapa de configuração do disco, foi observado que seguir as sugestões padrão do instalador é uma escolha adequada. Em uma tentativa inicial, foi feita uma alocação excessiva de espaço para o diretório `home`, o que posteriormente resultou em erros durante a instalação e configuração dos pacotes do sistema. Como consequência, foi necessário reinstalar o Minix, dessa vez adotando uma distribuição de espaço mais equilibrada entre as partições sugeridas pelo instalador.

Outro problema enfrentado ocorreu ao tentar clonar um repositório do GitHub diretamente no ambiente do Minix utilizando HTTPS. Ao executar o comando descrito abaixo (Listing 1), foi retornado o seguinte erro (Listing 2):

```
1 git clone https://github.com/joahannes/minix
```

Listing 1. Comando para clonagem do repositório MINIX.

```
1 fatal: unable to access 'https://github.com/joahannes/minix/':  
2 SSL certificate problem: unable to get local issuer certificate
```

Listing 2. Erro de certificado SSL retornado pelo Git.

Para contornar esse problema, foi utilizada a autenticação via SSH em vez de HTTPS, o que permitiu realizar o clone do repositório com sucesso.

2.2. Mudança dos banners do kernel

Para que as alterações fossem efetivas, foi necessário identificar corretamente os arquivos responsáveis por cada mensagem. A alteração da mensagem exibida durante o boot foi realizada no arquivo `/usr/src/minix/minix/kernel/main.c`. Nesse caso, a modificação foi feita diretamente no código-fonte do kernel, seguida da recompilação. O processo ocorreu sem dificuldades, e a nova mensagem foi exibida corretamente na inicialização do sistema.

De forma semelhante, a modificação da mensagem exibida no momento do desligamento (comando `poweroff`) também foi realizada no arquivo `/usr/src/minix/minix/kernel/main.c`. Após recompilar o kernel, a alteração foi aplicada com sucesso, sendo exibida corretamente durante o encerramento do sistema.

Já a modificação da mensagem exibida após o login apresentou maior dificuldade. Inicialmente, houve incerteza quanto ao arquivo responsável por essa mensagem. A primeira tentativa consistiu em alterar o arquivo `/usr/src/minix/etc/motd`, porém a modificação não surtiu efeito. Para resolver esse problema, foi utilizado o comando:

```
1 find / -name motd 2>/dev/null
```

Esse comando retornou dois arquivos relevantes: `/usr/src/minix/etc/motd` e `/etc/motd`. A partir disso, foi possível identificar que o sistema utiliza `/etc/motd` como fonte da mensagem exibida após o login. Após modificar também esse arquivo, a mensagem passou a ser exibida corretamente. Por fim, concluiu-se que nem todas as mensagens do sistema estão definidas no kernel. A mensagem exibida após o login é configurada por meio do arquivo `/etc/motd`.

Durante o processo, também foi enfrentado um problema ao utilizar o editor de texto, no qual percebemos que não conseguíamos abrir arquivos corretamente. Inicialmente, a causa do erro não era clara, mas após investigação foi identificado que o problema estava na falta de espaço de armazenamento no sistema. Esse esgotamento de espaço ocorreu devido às múltiplas recompilações do kernel, que geraram arquivos intermediários e ocuparam grande parte do disco disponível. A solução adotada foi a utilização do comando `make clean`, que remove arquivos temporários gerados durante o processo de compilação. Após a liberação de espaço, o editor voltou a funcionar normalmente, permitindo a continuidade das alterações.

2.3. Modificando o código `exec.c`

Durante a análise do código-fonte, observou-se que a chamada `exec` é implementada de forma distribuída entre o Process Manager (PM) e o Virtual File System (VFS). No PM, o caminho do executável é representado apenas como um endereço de memória (`vir_bytes`), o que impede sua utilização direta como string. Já no VFS, após a chamada da função `fetch_name`, o caminho do executável é convertido para uma string válida (`fullpath`), sendo este o local adequado para realizar a modificação. A alteração foi realizada no arquivo `exec.c` do VFS, inserindo uma chamada à função `printf` após a obtenção do caminho completo do executável.

Entretanto, foram encontrados problemas relacionados ao processo de compilação. Em tentativas iniciais, mesmo após utilizar `make` e `make install`, as alterações não eram refletidas no sistema em execução. Após investigação, concluiu-se que o ambiente estava inconsistente devido a compilações anteriores. A solução adotada foi restaurar uma snapshot da máquina virtual para um estado anterior, no qual nenhuma recompilação havia sido realizada. A partir desse ponto, a Atividade 2.2 foi refeita e recompilada corretamente pela primeira vez. Em seguida, a modificação referente à chamada `exec` foi implementada e o sistema recompilado novamente, resultando no funcionamento esperado. Assim, a atividade evidenciou não apenas o funcionamento interno da chamada `exec` e a arquitetura baseada em servidores do Minix, mas também a importância de um processo de compilação consistente e controlado ao modificar componentes do sistema operacional.

3. Etapa 2

3.1. Escalonamento de Processos

O escalonamento de processos é o coração dos sistemas operacionais multiprogramados. Quando vários processos disputam a CPU simultaneamente, cabe ao escalonador decidir quem executa a seguir, guiado por um algoritmo que tenta extrair o máximo de um recurso sempre escasso: o tempo do processador. Toda troca de processo, porém, tem um custo. A troca de contexto exige salvar o estado do processo atual, carregar o do próximo e atualizar estruturas como caches e tabelas de memória.

Para tomar boas decisões, o escalonador precisa entender o comportamento dos processos. A maioria alterna entre surtos de computação e pausas aguardando E/S, dividindo-os em CPU-bound, que ficam mais tempo calculando, e I/O-bound, que passam boa parte do tempo esperando. O sistema pode agir de forma preemptiva, interrompendo o processo em execução, ou não preemptiva, aguardando que ele libere a CPU. Como cada ambiente impõe prioridades distintas, seja maximizar o uso da CPU, reduzir o tempo de resposta ou cumprir prazos, não existe um algoritmo universalmente ideal.

3.2. Descrição do algoritmo de escalonamento padrão do Minix

O Minix 3 organiza o escalonamento em dezesseis filas de prioridade, onde tarefas de clock e sistema ocupam a mais alta, seguidas por drivers, servidores e, por último, processos de usuário. A fila de menor prioridade é exclusiva do processo IDLE. O escalonador sempre executa o processo na cabeça da fila não vazia de maior prioridade, usando round-robin dentro de cada fila. Processos que esgotam o quantum vão ao fim da fila; os que bloqueiam antes retornam à cabeça com o tempo restante, garantindo resposta rápida a operações de IO. Drivers e servidores recebem quantum grande mas normalmente bloqueiam voluntariamente antes de consumi-lo — o quantum é apenas proteção contra mau funcionamento.

O diferencial do Minix em relação a um round-robin simples é o ajuste dinâmico de prioridade. Um processo que esgota o quantum sem que nenhum outro consiga executar no intervalo é interpretado como loop e rebaixado a uma fila inferior. O inverso também ocorre: processos que consomem o quantum sem prejudicar os demais podem ser promovidos. Com componentes do sistema sempre em filas superiores, processos de usuário só executam quando o sistema não tem mais nada a fazer.

3.3. Round-Robin (Chaveamento Circular)

Seu funcionamento é baseado em uma fila circular: o processo na cabeça da fila é executado e, ao ter seu quantum esgotado ou ao transitar do estado bloqueado para pronto, é inserido na cauda, fazendo com que todos os processos se revezem de maneira ordenada e cíclica.

O algoritmo levanta duas questões importantes. A primeira é a escolha do valor do quantum: um valor muito pequeno faz o processador gastar mais tempo em trocas de contexto do que executando processos, enquanto um valor muito grande faz o Round Robin se comportar como o FCFS. A segunda é o que fazer com o quantum restante quando um processo se bloqueia antes de esgotá-lo: o próximo processo deve consumir apenas o tempo restante ou receber um quantum inteiro? Essa decisão afeta o comportamento e a previsibilidade do algoritmo.

3.3.1. Implementação

A implementação exigiu alterações em apenas um arquivo: `servers/sched/schedule.c`. Isso foi possível pela arquitetura de microkernel do sistema, que já separa a política de escalonamento (responsabilidade do servidor `sched`) do mecanismo (responsabilidade do kernel). Assim, como a lógica está isolada nesse único arquivo, a implementação se resume a garantir que nenhum processo de usuário tenha sua prioridade alterada.

3.3.2. Modificações Realizadas

1. Função `do_noquantum`

Esta função é chamada quando um processo esgota seu quantum de tempo. O comportamento original rebaixava a prioridade do processo a cada quantum esgotado. Assim, esse bloco foi removido para que o processo mantenha sua prioridade em `USER_Q`.

```
1 if (rmp->priority < MIN_USER_Q) {  
2     rmp->priority += 1;  
3 }
```

2. Função `do_nice`

Esta função é chamada quando um processo solicita uma mudança de prioridade via chamada `nice()`. No Round Robin, todos os processos de usuário devem ter a mesma prioridade sempre. Por isso, adicionamos uma verificação que impede qualquer alteração de prioridade para processos de usuário:

```
1 if (rmp->max_priority == USER_Q)  
2     return OK;
```

A verificação usa o campo `max_priority`, que é o identificador estável do tipo do processo (todos os de usuário tem `max_priority == USER_Q` garantido). O Process Manager inicializa o escalonador em espaço de usuário através da função `sched_init()`, localizada em `minix/servers/pm/main.c`. Essa função chama `sched_start()`, em `minix/servers/pm/schedule.c`, passando `USER_Q` como prioridade máxima. É através de `sched_init()` que podemos garantir que o processo `init` é o primeiro processo de usuário a ser escalonado e que todos os outros processos de usuário herdam a partir dele. Como a prioridade máxima de `init` é definida como `USER_Q`, todos os processos de usuário que descendem dele também terão prioridade máxima. A função `sched_start()` envia a mensagem 'SCHEDULING_START' ao servidor de escalonamento, que a encaminha para `do_start_scheduling()`, em `minix/servers/sched/schedule.c`. Esta função, então, atribui ao processo a prioridade máxima contida na mensagem:

```
1 rmp->max_priority = m_ptr->m_lsys_sched_scheduling_start.maxprio;
```

3.4. Escalonamento por Loteria (*Lottery Scheduling*)

O Lottery Scheduling adota uma abordagem probabilística onde cada processo recebe bilhetes que definem suas chances de obter a CPU. A cada decisão de escalonamento, sorteia-se um bilhete e o processo vencedor é executado. Sua principal característica é a justiça proporcional: um processo com o dobro de bilhetes recebe, em média, o dobro de tempo de CPU, sem risco de inanição.

3.4.1. Implementação

Foram realizadas alterações em `kernel/proc.h` e `kernel/proc.c`. As constantes `USER_Q` e `NR_SCHED_QUEUES`, obtidas via `#include <minix/config.h>`, delimitam diretamente as filas de usuário, tornando desnecessária qualquer verificação adicional de tipo de processo.

3.4.2. Modificações Realizadas

1. `kernel/proc.h`: adicionado o campo `p_tickets` à struct `proc` para armazenar os bilhetes de cada processo.

2. `kernel/proc.c`: definida a constante `DEFAULT_TICKETS = 10` e inicializado `p_tickets` em `proc_init()`. Como o kernel não possui `rand()`, foi implementado um gerador linear congruente estático:

```
1 static unsigned long lottery_seed = 12345UL;
2 static int lottery_rand(int total) {
3     lottery_seed = lottery_seed * 1103515245UL + 12345UL;
4     return (int)((lottery_seed >> 16) % (unsigned long)total) + 1;
5 }
```

A função `pick_proc()` foi modificada para implementar o sorteio apenas nas filas de usuário. Quando não há processos de usuário prontos, o comportamento original é mantido. Caso contrário, acumula-se o total de bilhetes, sorteia-se um número vencedor e percorrem-se as filas até que a contagem acumulada o alcance:

```
1 for (q = USER_Q; q < NR_SCHED_QUEUES; q++)
2     for (rp = rdy_head[q]; rp != NULL; rp = rp->p_nextready)
3         total_tickets += rp->p_tickets;
4
5 int winning = lottery_rand(total_tickets);
6 int count = 0;
7 for (q = USER_Q; q < NR_SCHED_QUEUES; q++) {
8     for (rp = rdy_head[q]; rp != NULL; rp = rp->p_nextready) {
9         count += rp->p_tickets;
10        if (count >= winning) {
11            if (priv(rp)->s_flags & BILLABLE)
12                get_cpulocal_var(bill_ptr) = rp;
13            return rp;
14        }
15    }
16 }
```

3.5. Garantido

O escalonamento garantido é baseado numa promessa simples: se há N processos rodando, cada um tem direito a $1/N$ do tempo de CPU. O algoritmo rastreia quanto cada processo realmente recebeu e sempre escolhe o mais prejudicado — aquele cuja razão entre CPU recebida e CPU prometida é a menor. Assim, a decisão de qual processo roda é feita pelo critério:

$$\text{razão} = \frac{\text{cpu_recebida}}{\text{cpu_prometida}}$$

O processo com menor razão é o mais prejudicado e tem prioridade para executar.

3.5.1. Implementação

A implementação do algoritmo Garantido no Minix 3 exigiu alterações em dois níveis distintos do sistema: no kernel, através dos arquivos `kernel/proc.h` e `kernel/proc.c`, e no servidor de escalonamento, através de `servers/sched/schedule.c`. Essa divisão reflete a arquitetura de microkernel do Minix — o kernel gerencia as filas de prontos e a contabilização de recursos, enquanto o servidor `sched` define a política de decisão.

3.5.2. Modificações Realizadas

1. Adição de novos campos na struct `proc` em `proc.h`, logo após `p_dequeued`

```
1 clock_t p_guaranteed_start; /* uptime quando o processo entra pela 1a
   vez na fila */
2
3 u32_t p_guaranteed_cpu; /* ticks de CPU efetivamente recebidos */
```

O `p_guaranteed_start` serve para calcular quanto de CPU foi prometido desde a criação do processo. O `p_guaranteed_cpu` acumula o quanto ele realmente recebeu.

2. Inicialização em `enqueue()` no `proc.c`

```
1 if (rp->p_guaranteed_start == 0) {
2     rp->p_guaranteed_start = get_monotonic();
3     rp->p_guaranteed_cpu = 0;
4 }
```

Na primeira vez que o processo entra na fila de prontos, os campos são inicializados, e a verificação `== 0` garante que a inicialização ocorra apenas uma vez.

3. Contabilização em `dequeue()` no `proc.c`

```
1 rp->p_accounting.dequeues++;
2
3 if (rp->p_guaranteed_start != 0)
4     rp->p_guaranteed_cpu++; /*conta mais um tick de CPU recebido */
```

Toda vez que um processo sai da fila (por bloqueio, fim de quantum ou preempção), incrementa o contador de CPU recebida. A verificação `!= 0` garante que apenas processos já inicializados como processos de usuário tenham seu uso de CPU contabilizado, evitando interferência de processos de sistema ou entradas vazias da tabela de processos.

4. Substituição de `pick_proc()` no `proc.c`

Esta é a alteração central. A função original simplesmente retornava o primeiro processo da fila mais prioritária. A nova lógica mantém esse comportamento para filas de sistema (prioridades 0 a `USER_Q - 1`) e aplica o critério garantido apenas para filas de usuário (`USER_Q = 7` a `MIN_USER_Q = 14`):

$$\text{cpu.prometido} = \frac{\text{uptime.atual} - \text{p.guaranteed.start}}{\text{n.processos.prontos}}$$

$$\text{razão} = \frac{\text{p_guaranteed_cpu} \times 1000}{\text{cpu_prometido}}$$

O processo com menor razão entre todos os prontos nas filas de usuário é escolhido para executar. A multiplicação por 1000 evita aritmética de ponto flutuante no kernel.

3.6. Resultados e Discussões

Todos os testes foram executados na máquina virtual do computador do Daniel Monteiro, com as seguintes especificações:

- **Processador:** 13th Gen Intel Core i7-13650HX, 2600 MHz, 14 Cores, 20 Processadores Lógicos;
- **Memória RAM:** 24.0 GB;
- **Armazenamento:** SSD NVMe 954 GB;

Os resultados foram coletados em ambiente virtualizado VirtualBox, o que impõe limitações à interpretação dos dados. Variações no tipo de armazenamento, quantidade de memória alocada à VM e carga concorrente do host influenciam diretamente os tempos medidos — especialmente no cenário IO-bound, onde o gargalo dominante é o acesso ao disco virtualizado. Por esse motivo, a análise foca no comportamento relativo entre os escalonadores. Os gráficos, código e resultados estão disponíveis no repositório GitHub.

3.6.1. Gráficos e Resultados

Para cada número de processos houveram 3 execuções de testes, para 10, 25, 50 e 100 processos. Dessa forma, tiramos a média para cada quantidade e algoritmo, isso tornou as médias de cada algoritmo mais fiéis e condizentes. Isso permitiu também a possibilidade de demonstrar o desvio padrão nos testes. A quantidade de operações foram, 100 mil para IO-Bound Ops e 100 milhões para CPU-Bound Ops. Assim, por meio de um código em python gerado pela LLM Claude, foi possível plotarmos tais gráficos:

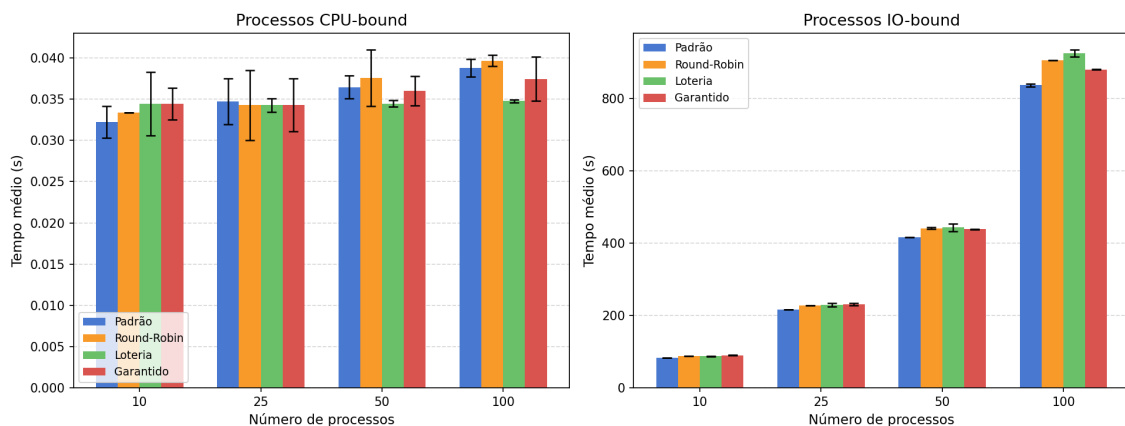


Figura 1. Resultados dos testes de escalonamento de processos.

3.6.2. Round-Robin (Chaveamento Circular)

Para processos CPU-bound, o Round-Robin apresentou desempenho similar ao padrão, com diferenças inferiores a 5 por cento em todos os cenários. Esse comportamento é esperado, pois processos CPU-bound raramente bloqueiam e tendem a consumir o quantum por completo, reduzindo o impacto da política de escalonamento. Para processos IO-bound, a diferença foi mais expressiva, chegando a aproximadamente 18 por cento com 100 processos. O escalonador padrão, ao promover automaticamente processos recém-desbloqueados de IO para filas de maior prioridade, reduz o tempo ocioso entre operações. O Round-Robin, por tratar todos os processos uniformemente em rodízio fixo, penaliza processos IO-bound que precisam de acesso rápido à CPU após o desbloqueio — efeito que se intensifica proporcionalmente ao número de processos concorrentes.

3.6.3. Escalonamento por Loteria

Para processos CPU-bound, o Lottery apresentou desempenho similar aos demais escalonadores nos cenários de 10 e 25 processos. Nos cenários de 50 e 100 processos, o Lottery passou a exibir os menores tempos de retorno. Esse comportamento se explica pelo fato de que, no escalonador padrão, processos CPU-bound são progressivamente rebaixados para filas de menor prioridade ao esgotarem seus quanta. O Lottery neutraliza esse efeito: ao percorrer todas as filas de usuário com bilhetes iguais em `pick_proc()`, um processo rebaixado mantém a mesma probabilidade de ser sorteado que qualquer outro, resultando em distribuição mais uniforme do tempo de CPU à medida que a carga aumenta. Vale mencionar que o mecanismo de aging do Minix permanece ativo em paralelo ao sorteio, adicionando um overhead que cresce com o número de processos. Para processos IO-bound, o Lottery apresentou os maiores tempos de retorno. O escalonador padrão mantém processos recém-desbloqueados em filas de maior prioridade, reduzindo o tempo ocioso entre operações. O Lottery, ao atribuir bilhetes iguais a todos os processos independentemente do histórico de bloqueios, não concede essa vantagem a processos IO-bound após o desbloqueio, efeito que se intensifica com o número de processos concorrentes.

3.6.4. Escalonamento Garantido

Para processos CPU-bound, o Garantido apresentou tempo médio superior ao padrão no cenário de 10 processos, revertendo esse comportamento nos outros. O desvio padrão baixo e consistente entre as três execuções indica estabilidade do algoritmo, e à medida que o número de processos cresce, o critério de menor razão passou a diferenciar melhor os concorrentes, permitindo decisões de escalonamento eficientes e reduzindo o tempo de retorno em relação ao padrão. Para processos IO-bound, o Garantido apresentou tempo médio igual ou ligeiramente superior ao padrão em todos os cenários. Processos IO-bound bloqueiam frequentemente e retornam à fila de prontos com pouca CPU acumulada, resultando em razão baixa e levando o algoritmo a detectá-los como prejudicados. Assim, estes são priorizados no retorno do bloqueio, comportamento parecido com o ajuste dinâmico de prioridade entre filas do padrão Minix, que também os favorece. Logo, os dois algoritmos produzem tempos similares nesse tipo de carga, e a pequena diferença reflete o overhead da dupla varredura das filas a cada decisão de escalonamento.

4. Conclusão

Este projeto permitiu explorar na prática os conceitos de escalonamento de processos em um sistema operacional real, o MINIX 3, desde sua instalação e configuração em ambiente virtualizado até a implementação e avaliação de três algoritmos distintos. A experiência evidenciou que a escolha do algoritmo de escalonamento tem impacto direto no desempenho do sistema, especialmente em cargas IO-bound, onde o escalonador padrão do MINIX demonstrou vantagem ao promover processos recém-desbloqueados, comportamento que os algoritmos Round-Robin e Loteria não reproduzem, enquanto o Garantido aproxima-se dele por meio da compensação proporcional de CPU. Para cargas CPU-bound, todos os algoritmos convergiram para desempenhos similares, indicando que o gargalo nesse perfil é o tempo de processamento e não a política de seleção. Por fim, a arquitetura de microkernel do MINIX 3 mostrou-se um ambiente didático valioso, pois isola a política de escalonamento do mecanismo, permitindo modificações cirúrgicas e seguras sem comprometer a estabilidade do sistema.

5. Detalhamento da divisão do trabalho em equipe

Para organizar o trabalho do grupo, foram agendadas reuniões semanais com o objetivo de sincronizar o progresso de cada integrante e alinhar as decisões tomadas ao longo do projeto. Na etapa referente à implementação dos algoritmos de escalonamento, cada integrante ficou responsável por estudar individualmente uma opção de algoritmo e apresentar uma explicação sobre ela na reunião semanal seguinte. A partir dessas apresentações, o grupo escolheu em conjunto três algoritmos a serem implementados: Round Robin, Loteria e Garantido.

Após a escolha dos algoritmos, o grupo foi dividido em duplas, e cada dupla ficou responsável por um algoritmo. Daniel e Lael ficaram responsáveis pelo Round Robin, Henrique e Lucas pelo Loteria, e Camila e João Gomes pelo Garantido. Cada dupla foi encarregada de compreender a implementação atual do Minix 3 relacionada ao seu algoritmo e de estudar o algoritmo em si, de forma a implementá-lo da maneira mais correta possível.

Referências

- da Costa, J. B. D. (2026). Capítulo 2 - processos e threads - aula 11 - UC 2612 sistemas operacionais. Slides de aula. Disponível em: [joahannes.github.io](https://github.com/joahannes).
- Oliveira, G. B. (2021). Escalonamento no Minix 3 - sistemas operacionais. YouTube. Disponível em: <https://youtu.be/-TUmEsa3sno>. Acessado em: 30 de maio de 2026.
- Tanenbaum, A. S. and Bos, H. (2016). *Sistemas Operacionais Modernos*. Pearson, 4 edition.
- Tanenbaum, A. S. and Woodhull, A. S. (2006). *Sistemas Operacionais: Projeto e Implementação*. Bookman Editora, 3 edition.